

Data Compression Project (Self-Paced Stages)

Dr. Faisal Aslam

CSE 4/562 | Complete by End of Semester

20% of Final Grade | Complete All 3 Stages
You control the pace: each stage takes as long as you need

1 Project Structure

This project has **3 stages**. You must complete all stages by the end of semester, but you control how much time to spend on each. Recommended effort: **30-40 hours total**.

- **Stage 1: Core Implementation** — 10-15 hours
- **Stage 2: Full Pipeline** — 10-15 hours
- **Stage 3: Analysis & Report** — 10 hours

2 Choose Your Track

Pick **one** track based on your interest:

Track A: LZSS + Arithmetic

- Dictionary-based
- More intuitive
- Good for text files

Track B: BWT + MTF + Arithmetic

- Transform-based
- More mathematical
- Better compression usually

3 Stage 1: Core Implementation

Goal

Get the core compression algorithm working, even without entropy coding.

Checkpoint Requirements

- Working encoder/decoder for your chosen algorithm
- Can compress and decompress a small file (under 10KB)
- Unit tests for key functions
- 2-page report explaining your implementation

Track A: LZSS Core

Algorithm 1 LZSS with Linear Search (Simplified)

```
1: procedure COMPRESS(input[0..n - 1])
2:   i ← 0
3:   while i < n do
4:     (match_len, distance) ← FindMatch(input, i)
5:     if match_len ≥ 3 then
6:       Output flag 1
7:       Output match_len (4 bits)
8:       Output distance (12 bits)
9:       i ← i + match_len
10:    else
11:      Output flag 0
12:      Output input[i] (8 bits)
13:      i ← i + 1
14:    end if
15:  end while
16: end procedure
```

FindMatch function: Scan backwards up to 8KB, find longest match of at least 3 bytes.

Track B: BWT Core

Algorithm 2 Naive BWT

```
1: procedure BWT( $S[0..n-1]$ )
2:    $T \leftarrow S + \#$  ▷ Add unique marker (assume not in input)
3:   Create list of all  $n + 1$  rotations of  $T$ 
4:   Sort rotations lexicographically
5:    $L \leftarrow$  last character of each rotation
6:   Find  $primary =$  index where original string appears
7:   return  $(L, primary)$ 
8: end procedure
9: procedure INVERSEBWT( $L[0..n], primary$ )
10:  Create empty table of size  $(n + 1) \times 2$ 
11:  Add  $(L[i], i)$  as first column
12:  Sort rows by first column  $\rightarrow$  get  $F$ 
13:  Build next array:  $next[i] =$  original position of  $L[i]$ 
14:   $row \leftarrow primary$ 
15:  for  $i \leftarrow 0$  to  $n$  do
16:    Prepend  $L[row]$  to result
17:     $row \leftarrow next[row]$ 
18:  end for
19:  Remove marker from result
20:  return result
21: end procedure
```

What to Submit for Stage 1

1. Source code with clear documentation
2. Unit tests showing correctness
3. 2-page PDF report including:
 - Algorithm explanation in your own words
 - Challenges you faced and how you solved them
 - Sample output on a small test file

4 Stage 2: Full Pipeline

Goal

Add arithmetic coding to create a complete compressor.

Checkpoint Requirements

- Arithmetic coder integrated with your core algorithm
- Full pipeline works on files up to 1MB
- Can compress and decompress correctly
- 3-page report with initial results

Arithmetic Coding (Simplified)

Algorithm 3 Static Arithmetic Coder

```
1: procedure ENCODER(symbols[0..m - 1])
2:   Use fixed probabilities from training file
3:   low  $\leftarrow$  0, high  $\leftarrow$  1
4:   for each symbol s do
5:     range  $\leftarrow$  high - low
6:     high  $\leftarrow$  low + range  $\times$  cum_prob[s]
7:     low  $\leftarrow$  low + range  $\times$  cum_prob[s - 1]
8:     while high < 0.5 or low > 0.5 do
9:       Output bits, rescale
10:    end while
11:  end for
12: end procedure
```

Probability Models

Maintain **three separate models**:

1. **Token type** (Track A: flag; Track B: MTF symbol)
2. **Literal bytes** (256 symbols)
3. **Match parameters** (length + distance for Track A)

Implementation Options

Choose based on your confidence:

Easier	More Advanced
Static probabilities (fixed)	Adaptive (counts update)
32-bit integers	64-bit precision
One model for everything	Separate models per context
No EOF handling	Proper EOF marker

What to Submit for Stage 2

1. Updated source code with arithmetic coding
2. Test results on at least 3 files (text, binary, repetitive)
3. 3-page report including:
 - How you implemented arithmetic coding
 - Compression ratios achieved
 - Comparison against raw (no entropy coding)
 - Challenges faced

5 Stage 3: Analysis & Report

Goal

Evaluate your compressor against standard tools and write final report.

Required Comparisons

Test your compressor against `gzip` on these files:

1. **English text:** `alice29.txt` from Canterbury Corpus
2. **DNA sequence:** *E. coli* genome (FASTA)
3. **Source code:** Any `.c` or `.py` file >100KB
4. **Repetitive:** 500KB of repeating pattern
5. **Random:** 100KB from `/dev/urandom`

Metrics to Measure

For each file, report:

Metric	Description
Original size	File size in bytes
Compressed size	Your output file size
gzip size	Run <code>gzip -9 file</code> and check size
Compression ratio	Original / Compressed
Time	Use <code>time</code> command

Final Report Structure

Title Page: Name, track, date

1. Introduction (1 page)

- Which track you chose and why
- Overview of your approach

2. Implementation (2-3 pages)

- Key algorithms with pseudocode
- Important design decisions
- Challenges and solutions

3. Results (2 pages with graphs)

- Bar chart: Your ratio vs `gzip` for all files
- Table with all measurements
- Analysis: Why does your compressor win/lose on each file?

4. Reflection (1 page)

- What would you do differently?
- What did you learn?
- How many hours did each stage take?

What to Submit for Stage 3

1. Final source code (clean, documented)
2. Final report (PDF, 6-8 pages)
3. All test files and results

6 Grading Rubric

Stage	Requirement	Points
Stage 1: Core	Working encoder/decoder	5
	Unit tests	2
	Stage 1 report	3
	Stage 1 Total	10
Stage 2: Pipeline	Arithmetic coder integrated	5
	Works on 3 file types	3
	Stage 2 report	2
	Stage 2 Total	10
Stage 3: Analysis	Comparison against gzip (5 files)	5
	Graphs and analysis	3
	Final report quality	2
	Stage 3 Total	10
Overall Total		30

Bonus Points (+2 max)

- Adaptive probability models (instead of static)
- Beats gzip on any file
- Clean, well-documented code with Makefile
- Creative optimization